

Homework 22: Research It Yourself

The last homework in this curriculum is not about writing Java. It is about answering "can I use this?" from primary sources, because every other skill here has a shelf life and this one does not.

Part One: Three Features, Researched

Create `FeatureResearch.java` in the classic form.

Pick three features covered in this curriculum, at least one **Final** and at least one **Preview**. For each, find and record from openjdk.org:

Field	Where to find it
JEP number	the JEP index, or the release page
Status	the JEP's own header
Delivering release	the Release field
Is that release an LTS?	the release schedule

Print them as a table, and **print the URLs you used** underneath. A claim with no source does not count here.

Part Two: The Three Questions

Write a method applying the module's checklist to a feature:

1. Is it Final? If not, which round?
2. Which release delivered it, and is that an LTS?
3. What is the minimum version you must support?

It must print a verdict. Run it against a Final feature and a Preview one, assuming you must support **JDK 21**.

The Interesting Result

One of your two will be Final, shipped in an LTS, and **still unusable**.

Write a comment explaining why. Your answer must distinguish a feature being **safe** from a feature being **available**, because conflating those is how somebody ships a library nobody can depend on.

One implementation note. Take the versions as numbers, not strings. Comparing `"JDK 21"` with `"JDK 25"` as text happens to work; comparing `"JDK 9"` with `"JDK 25"` does not. Say in a comment why that is the same mistake as parsing `java.version`.

Part Three: Read A History Section

Pick a feature that has previewed more than twice. Open its JEP and find the **History** section.

List every round: JEP number, release, and whether it was an incubator or a preview.

Then Answer

- How many rounds in total, counting incubator ones?
- Did the JEP **number** change between rounds, and what does that tell you that a repeated number would not?
- Did anything surprise you about the history compared with what you assumed before reading it?

That last question is the point of the exercise. Write down what you expected and what you found.

Then compare with a feature that finalised quickly, and state the heuristic you would use in future.

Part Four: Check The Version Properly

Print `Runtime.version()`, its `feature()`, and the `java.version` property.

Then write a check for a minimum required version.

Explain It

A comment on why parsing `java.version` is a trap. Be specific: what was the format before Java 9, what is it now, which component meant the version number in each, and what happened to code that split on dots and took index 1.

Acceptance Criteria

- ☐ Runs with `java FeatureResearch.java`
- ☐ Three features with JEP number, status, release and LTS status
- ☐ The source URLs are printed
- ☐ The three-question method prints a verdict for a Final and a Preview feature
- ☐ One feature is Final, in an LTS, and still unusable, with the reason explained
- ☐ The safe-versus-available distinction is stated explicitly
- ☐ Versions are compared as numbers, with a comment on why
- ☐ A full preview history is listed, incubator rounds included
- ☐ The JEP-number-changing signal is explained
- ☐ What you expected versus what you found is written down
- ☐ A heuristic for future features is stated

☐ `Runtime.version()` is used, with the `java.version` trap explained

Stretch

Find a feature that was **withdrawn** after previewing, rather than finalised.

Write a comment on what its JEP says about why, and what that implies for anyone who had shipped code using it during the preview.

Hint, if you cannot find the History section

It is near the bottom of the JEP, below Alternatives and Risks. It is the most useful part of the document and the one nobody scrolls to.